



Technical guide: Data version control

datos.gob.es

November 2025



VICEPRESIDENCIA
PRIMERA DEL GOBIERNO
MINISTERIO
DE ASUNTOS ECONÓMICOS
Y TRANSFORMACIÓN DIGITAL

SECRETARÍA DE ESTADO
DE DIGITALIZACIÓN
E INTELIGENCIA ARTIFICIAL

red.es Iniciativa **aporta**

datos.gob.es
reutiliza la información pública

This document has been prepared within the framework of the Aporta Initiative (datos.gob.es), developed by the Ministry for Digital Transformation and Public Function through the Public Business Entity Red.es.

Legal notice: This work is licensed under a Creative Commons Attribution 4.0 International License (CC BY 4.0). Its reproduction, distribution, public communication, and transformation to create derivative works are permitted without restriction, provided that the copyright holder (Ministry for Digital Transformation and Public Administration through the Public Business Entity Red.es) is cited. The full license can be consulted at: <https://creativecommons.org/licenses/by/4.0>

Acknowledgments:

Credit for the first edition of this document goes to:

- Adrian Rico Alonso.

For their contributions, suggestions and availability in the preparation of this guide to:

- David Portolés Rodríguez (Expert in geospatial data and linked open data)

TABLE OF CONTENTS

1. DATA VERSION CONTROL	6
1.1. Fundamentals of Data Versioning.....	6
1.1.1. History and evolution.....	6
1.1.2. Fundamental differences with code versioning.....	7
1.1.3. Unique characteristics of data as a digital asset	9
1.2. Current problems in data projects.....	9
1.2.1. Detailed Problem Cases: Three Real-World Scenarios	9
1.2.2. Costs of not having version control	11
1.2.3. Impact on reproducibility and collaboration	11
1.3. Fundamental concepts and terminology	12
1.3.1. Snapshots	12
1.3.2. Data Lineage.....	12
1.3.3. Metadata Tracking	13
1.3.4. Checksums and data integrity.....	13
1.3.5. Data Provenance vs. Data Lineage.....	13
1.4. Critical application cases.....	14
1.4.1. Machine Learning models in production	14
1.4.2. Regulatory audits	14
1.4.3. Reproducible scientific research	15
1.4.4. Collaboration in distributed teams	15
2. VERSIONING STRATEGIES AND METHODOLOGIES	15
2.1. Semantic versioning adapted to data	15
2.1.1. Detailed definition of MAJOR.MINOR.PATCH for datasets.....	16
2.1.2. Specific examples of changes.....	16
2.1.3. Borderline cases and how to categorize them	17
2.1.4. Advanced naming conventions.....	17
2.1.5. Integration with version control tools	17
2.2. Versioning strategies.....	18
2.2.1. Full versioning (Full Snapshotting).....	18
2.2.2. Incremental Versioning (Delta Versioning).....	18

2.2.3. Hybrid techniques	19
2.2.4. Data deduplication and space optimization	19
2.2.5. Comparison of storage costs.....	19
2.3. Metadata management and traceability	20
2.3.1. What metadata to capture?	20
2.3.2. Metadata formats	21
2.3.3. Automation of metadata capture	21
2.3.4. Lineage Tracking: Automatic vs. Manual	21
2.4. Branching and merging for datasets.....	22
2.4.1. When to create branches in data?.....	22
2.4.2. Merge strategies for datasets	22
2.4.3. Data conflict resolution.....	23
2.5. Retention and archiving policies.....	23
2.5.1. Criteria for deciding which versions to maintain	23
2.5.2. Long-term archiving strategies	24
2.5.3. Costs vs. benefits of archiving.....	24
3.VERSIONING TOOLS AND TECHNIQUES IN PRACTICE	24
3.1.DVC (Data Version Control)	25
3.1.1.Installation inWindows	25
3.1.2. Initial configuration.....	26
3.1.3. All basic commands.....	27
3.1.4. Configuration of different types of remote storage	28
3.1.5. Data Pipelines with DVC.....	29
3.1.6.Integration with Git workflows.....	30
3.1.7. Troubleshooting common problems	31
3.2. Git LFS - An alternative for large files	32
3.2.1. When to use Git LFS vs DVC	32
3.3. Native cloud solutions.....	33
3.3.1. AWS S3 with native versioning	33
3.3.2. Google Cloud Storage (GCS) versioning	33
3.3.3. Azure Blob Storage options.....	34
3.3.4. Comparison of costs and functionalities.....	34
3.3.5.Integration with Machine Learning tools.....	34

3.4. Manual versioning and conventions.....	35
3.4.1. When is manual versioning appropriate?.....	35
3.4.2. Robust naming conventions.....	35
3.4.3. Templates for manual documentation	35
4. CASE STUDY PRACTICAL.....	36
4.1. Initial configuration.....	37
4.1.1. Minimum prerequisites.....	37
4.1.2. Initialization of repositories	37
4.1.3. Configuring local remote storage	38
4.2. First data versioning.....	38
4.2.1. Creation and execution of the data generator script	39
4.2.2. Version control with DVC and GIT	39
4.2.3. Documentation and synchronization with the remote	40
4.3. Update simulation.....	41
4.3.1. Creation and execution of the update script	41
4.3.2. Versioning of data updates and control updates	42
4.4. Recovery and rollback.....	42
4.5. Conclusion.....	43
5. BEST PRACTICES AND IMPLEMENTATION.....	44
5.1. Fundamental Conventions and Standards.....	44
5.2. Governance roles and processes	44
5.3. Compliance and retention considerations.....	45
5.4. Roadmap for progressive implementation	45
5.5. Tool Decision Matrix	46
6. CONCLUSION.....	46

1. DATA VERSION CONTROL

In the era of Big Data and artificial intelligence, data has become the most valuable asset for any organization. It fuels machine learning (ML) models, forms the foundation for business strategies, and provides the evidence that underpins scientific research. However, unlike other digital assets such as source code, data is fluid, complex, and often chaotic. It changes, grows, and evolves at a breakneck pace, presenting a fundamental challenge: how can we manage this evolution reliably, reproducibly, and collaboratively?

The answer lies in Data Version Control (DVC), an emerging discipline that adapts software management principles to the data ecosystem. It's not simply about making backups or naming files with dates. It's a systematic approach to recording every significant change to a dataset, enabling teams to understand its history, revert to previous states, and collaborate seamlessly. This section lays the conceptual groundwork for DVC, explores the critical problems it solves, and defines the essential terminology every data professional should know.

1.1. Fundamentals of data versioning

To understand the need for data version control, it's helpful to look at how more established disciplines have solved similar problems. Software engineering, for example, would have been unthinkable without tools like Git, which allow thousands of developers to collaborate on massive codebases. Data versioning draws inspiration from this revolution but adapts to the unique challenges presented by data.

1.1.1. History and evolution

The concept of version control isn't entirely new, but its formalization and the creation of specialized tools are relatively recent. Historically, data management was done manually and in a rudimentary way, an approach that quickly became unsustainable.

- **Phase 1: Manual versioning (The era of organized chaos).** The most basic and, unfortunately, still common method. It consists of saving multiple copies of a file with descriptive names:
 - sales_report_2023_final.csv
 - sales_report_2023_final_corrected.csv
 - sales_report_2023_final_final_v2_NOW_YES.csv

This strategy is error-prone, confusing, and doesn't scale. There's no central record of changes, it's impossible to tell what differentiates '_fixed' from '_v2', and collaboration is a recipe for disaster, with multiple "final versions" circulating via email.

- **Phase 2: Centralized storage with backups.** An improvement over the manual method. Data is stored on a central server or in the cloud (e.g., SharePoint, Google Drive), and regular backups are performed. This protects against data loss, but it's not a true version control system. Reverting to a previous version involves restoring a full backup, a slow and disruptive process. Furthermore, it doesn't record the reasons behind changes or allow for working on parallel branches or experiments.
- **Phase 3: Adapting coding tools (the brave attempt).** With the rise of data science, teams tried

directly applying tools like Git to datasets. They soon discovered its limitations. Git is optimized for small text files and lines of code. When faced with a 10GB CSV file, its performance degrades dramatically. Storing every version of a large dataset in Git would cause the repository to grow to an unmanageable size. Tools like Git LFS (Large File Storage) emerged as a workaround, storing pointers in Git and the large files elsewhere, but they still failed to address data granularity and lineage.

- **Phase 4: Native data version control tools.** The obvious need led to the development of specialized tools like DVC (Data Version Control), Pachyderm, and Delta Lake. These solutions are designed from the ground up for data. They understand that data is large and often binary. Instead of simply copying data within the repository, they manage metadata and pointers, integrating with optimized storage systems (like Amazon S3 and Google Cloud Storage) and leveraging Git for what it does best: versioning code and metadata.

1.1.2. Fundamental differences with code versioning

The reason Git doesn't work well for "unsupported" data is that code and data are fundamentally different. Thinking of them as the same thing is the root of many problems in data projects. Consider that versioning code is like editing a book manuscript. Changes are textual, incremental, and easy to compare (diff). Versioning data is like managing versions of a high-definition movie. The file is massive, opaque (binary), and a small change (correcting a single frame) creates a completely new file from a storage perspective. You don't compare "lines" of film; you compare the final result.

The following is a detailed comparative table highlighting these critical differences.

Feature	Code versioning (e.g. Git)	Data versioning (e.g., DVC)	Explanation and consequences
Size and scale	Generally small (KB to MB). Complete repositories rarely exceed a few GB.	From large to massive (MB to TB or PB). A single file can be much larger than an entire codebase.	Git stores every version of every file, which would cause the repository size to explode with data. CVD tools store data outside the repository, in storage optimized for large files.
Nature of the content	Primarily plain text (source code). Human-readable and easy to distinguish.	Often binary, compressed, or structured (images, CSV, Parquet, H5 patterns). Not readable or comparable line by line.	The `git diff` function is useless for a 50 GB Parquet archive. CVD focuses on differentiating versions at the metadata level (who changed what, when, why) and metrics, not on comparing the content byte by byte.

Unit of exchange	The line of code. The changes are granular and localized.	The entire dataset (or a significant subset). A change could be adding a row, correcting a column, or regenerating the entire dataset.	A code commit might be "Corrected typo in function X". A data commit might be "Added sales data for Q3/2025", which could involve changing millions of rows.
Immutability	Mutable by design. The code is constantly edited and refactored.	Often immutable or additive. Once data is generated (e.g., a transaction), it shouldn't change. "Updates" are usually new data or corrections that generate a new version.	The CVD mental model is based on immutable snapshots. A dataset is not edited; a new version is created from the previous one, preserving the history.
Storage	The entire history is stored locally in each contributor's .git directory.	Storage is remote and centralized (e.g., S3, GCS, Azure Blob Storage). Only the necessary data is downloaded locally.	Downloading the complete history of a 500 TB dataset to each data scientist's machine is impractical. CVD allows "lazy" or on-demand access to the data versions needed.
Lineage(Lineage)	Implicit in the commit history and branches. The lineage of the code is the code itself.	Critical and complex. The data lineage includes the original sources, each transformation script, and the generated models.	The CVD must record not only that dataset_v2.csv exists, but that it was created by running script_clean.py on dataset_v1.csv with the parameter threshold=0.95.
Merger operations	git merge is a common and powerful operation for combining text changes from different branches.	Data "fusion" is conceptually difficult and risky. How do you merge two versions of a customer dataset with conflicting corrections?	Instead of merging, data workflows typically involve regenerating a dataset from updated sources or choosing one branch over another. Automated merging is rare and risky.

Reproducibility	This is achieved by cloning the repository and executing the code in a specific commit.	It requires the code, the exact version of the data, and the computing environment. The lack of any of these elements breaks reproducibility.	The CVD is the piece that connects the code (in Git) with the exact state of the data it used, making experiments and models truly reproducible.
------------------------	---	---	--

1.1.3. Unique characteristics of data as a digital asset

Data is not just "big files". It has intrinsic characteristics that demand a specialized management approach.

1. **Data Gravity:**It's a term coined to describe the idea that as data accumulates in size, it tends to attract more applications and services. Moving a 100 TB dataset is expensive, slow, and risky. Therefore, it's easier to move the processing (code) to where the data resides. CVD systems respect this principle by keeping the data in a centralized, optimized storage and moving only the lightweight metadata files.
2. **Data Entropy:**Data systems naturally tend toward disorder. Without active governance, schemas change, quality degrades, and documentation becomes outdated. A CVD system acts as a force against entropy, enforcing structure, recording metadata, and ensuring that each version is well-defined and recoverable.
3. **Complex dependencies:**A single end result, such as a BI report or an ML model, can depend on a complex processing chain: raw data from an API, a cleaning script, an enrichment process that queries a database, and finally a training script. CVD not only versions the final file, but ideally captures this entire dependency pipeline, allowing the entire chain to be reproduced from beginning to end.

1.2. Current problems in data projects

The absence of a robust data version control system is not merely a technical inconvenience; it is a constant source of risk, inefficiency, and failure in data projects. When the question "What data was used to obtain this result?" cannot be answered with certainty, the entire foundation of the project becomes unstable.

1.2.1. Detailed problem cases: three real-world scenarios

These scenarios, based on real (anonymized) problems, illustrate the tangible consequences of poor version control.

Scenario 1: The Ghost Model

- **Situation:**A data science team at an e-commerce company develops a product recommendation model. After months of work, they achieve 92% accuracy in their tests and deploy it to production. For the first few weeks, it works perfectly, increasing sales by 15%.
- **The problem:**Two months later, the model's performance plummets. Recommendations are

irrelevant, and the click-through rate collapses. The team decides to retrain the model to adapt it to the new customer data. However, the original data scientist has left the company. Their successor finds the training script on Git, but the `training_data.csv` file on the shared server has been updated several times since the initial deployment with new marketing campaigns and data corrections. There is no record of which exact version was used for the original, successful model.

- **Consequences:** The team spends weeks trying to "guess" the original state of the data, testing different filters and preprocessing methods without success. They are unable to replicate the 92% accuracy. The company loses the competitive advantage of a 15% increase in sales, and trust in the data team erodes.
- **Solution with CVD:** With a CVD system, the Git commit containing the final version of the training script would have been linked to a specific hash of the data version. The new data scientist would simply have to run ``dvc pull`` to download the exact version of the data (from two months ago) and reproduce the original training environment in minutes, allowing them to analyze what had changed and why performance had degraded (model drift).

Scenario 2: The impossible audit

- **Situation:** A financial institution must submit quarterly regulatory reports on its risk exposure. These reports are generated using complex models that are fed with real-time market data (stock prices, interest rates, etc.).
- **The problem:** One year after submitting the Q2 report, regulators requested an audit and asked the entity to reproduce the exact figures from that specific report and justify its calculations. The team has the risk model code from that period. However, the market database has been continuously updated. The Q2 data from the previous year has been corrected by the vendor, and some time series have been adjusted. Recreating the exact state of the database at that time is a monumental, if not impossible, task.
- **Consequences:** The IT team spends weeks trying to restore backups and cross-reference logs to reconstruct the state of the data. The result is not 100% identical. The organization faces potential penalties for failing to guarantee the traceability of its reports, and its reputation suffers a severe blow.
- **Solution with CVD:** A workflow using CVD would take an immutable snapshot of the market data used for each quarterly report. This snapshot would be versioned and associated with the report's code. When an audit arrived, the team would simply retrieve the `report_risk_Q2_2024` version from their project, and the system would automatically download the corresponding market data snapshot, allowing the report to be regenerated with 100% accuracy within hours.

Scenario 3: Collaborative Chaos

- **Situation:** Two data analysts, Ana and Bruno, are working on the same project to segment customers. They both start with a common dataset: `customers_raw.csv`.
- **The problem:** Ana decides that customers without a last purchase date should be removed. She creates a script, generates `customers_cleaned_ana.csv`, and begins her analysis. Simultaneously, Bruno decides that customers without a last purchase date should be imputed using the mean.

He creates his own script, generates `customers_cleaned_bruno.csv`, and continues his work. A week later, a third team member, Carla, needs the clean dataset and finds both files. She doesn't know which one is the "official" one, what criteria were used in each, or if she can combine them. The situation worsens when Ana updates her file, accidentally overwriting Bruno's file in a shared folder.

- **Consequences:** Hours of work are lost. There is confusion and frustration within the team. Ana's and Bruno's analyses are not comparable because they are based on different cleaning assumptions. The lack of a single "source of truth" breeds distrust in the data.
- **Solution with CVD:** Ana and Bruno would create two data "branches," just like in Git. The main branch would contain the original dataset. Ana would create a branch called `feature/remove-null-customers` and Bruno one called `feature/input-null-customers`. Each would work on their isolated version. Their changes and the logic behind them would be documented in commit messages. Later, the team could review both strategies, compare them, and decide which one (or a combination) to incorporate into the main branch as the official, clean version of the dataset.

1.2.2. Costs of not having version control

The costs arising from these problems are not only technical, but also directly impact the business and mission of the organization.

- **Financial costs:**
 - **Lost work hours:** Data scientists and engineers, whose salaries are high, spend a disproportionate amount of time on "data archaeology" instead of generating value.
 - **Faulty business decisions:** Basing strategies on incorrect analysis or flawed models can lead to losses of millions.
 - **Regulatory fines:** The inability to demonstrate traceability in regulated sectors (finance, health) can lead to severe penalties.
- **Opportunity costs:**
 - **Slowing down of innovation:** If each new project involves reinventing the wheel to clean and validate the data, the pace of innovation slows down.
 - **Inefficient collaboration:** Teams become afraid to touch the data for fear of breaking something, creating silos and duplication of effort.
- **Reputation costs:**
 - **Loss of trust:** both internal (from management in the data team) and external (from clients or the scientific community).
 - **Damage to the brand:** A public error due to incorrect data can have a lasting impact on brand perception.

1.2.3. Impact on reproducibility and collaboration

- **Scientific reproducibility:** In science, reproducibility is the cornerstone of validity. An experiment that cannot be replicated by others is unreliable. In data science, this means that an analysis or model must be able to be exactly regenerated, yielding the same result. Without versioning the data, reproducibility is simply a utopia.
- **Team collaboration:** Modern teamwork is distributed and asynchronous. CVD provides a common

language and workflow. It allows team members to work in parallel, experiment with data safely in isolated branches, and merge their contributions in an orderly fashion, much like software developers collaborate on complex codebases. Without it, collaboration is limited to sending files via email, a practice that inevitably leads to chaos.

1.3. Fundamental concepts and terminology

To effectively implement and use a CVD system, understanding its vocabulary is crucial. These concepts form the grammar of data versioning.

1.3.1. Snapshots

A snapshot is a read-only, completely immutable copy of the state of a dataset at a specific point in time. It is the most fundamental concept of CVD.

- **Analogy:** Think of a document's "Save As..." function, but with superpowers. When you create a snapshot, you're not simply saving a copy. You're creating a permanent, identifiable version that can't be altered. Any future changes will require creating a new snapshot.
- **Practical example:**
 1. A team receives the January sales dataset: `sales_january.csv`.
 2. They decide to version it. Using a CVD tool, they create the first snapshot. The tool calculates a unique "digital fingerprint" (a checksum) for the file and stores it in a secure location (e.g., an S3 bucket). In their Git repository, only a small metadata file pointing to that snapshot is stored.
 3. Later, they discover errors in some transactions. They correct the file.
 4. Now, they create a second snapshot. The tool detects that the file has changed, calculates a new checksum, uploads the new version to storage, and creates a new metadata file.
 5. They now have two immutable and identifiable versions of the January dataset. They can switch between them with a single command, knowing that `version_1` will always be the original file and `version_2` will always be the corrected file.

1.3.2. Data Lineage

Data lineage is the complete record of the data lifecycle. It documents its origin, every transformation it has undergone, and where it has moved over time.

- **Analogy:** It is the family tree of a piece of data. It shows who its "parents" were (raw data sources), how it was "educated" (cleaning and transformation scripts), and what it "became" (final dataset, model, or report).
- **Practical example:** A good CVD system not only versions `model_churn.pkl`, but also records the entire preprocessing, training, and testing graph. This means that if someone questions the model, it can be traced back to see that it was generated with `train_model.py`, which in turn used `sales_clean.parquet`, which came from two original sources processed by `clean_sales.py`. If `sales_clean.parquet` changes, the system knows that `model_churn.pkl` is outdated and needs to be regenerated.

1.3.3. Metadata Tracking

Metadata is "data about data." Metadata tracking involves recording crucial contextual information about each version of the dataset.

- **Use case:** Imagine two versions of a model: model_v1 has 85% accuracy and model_v2 has 88% accuracy. Without metadata, we don't know why v2 is better. With metadata tracking, we can record:
 - **Version metadata:** who created it, when, and with what commit message ("New marketing campaign data added").
 - **Schema metadata:** the number of columns, their names, and data types. Perhaps v2 used a new column (feature) that improved performance.
 - **Descriptive metadata:** statistics such as the number of rows, the average of a key column, the number of null values.
 - **Evaluation metrics:** accuracy, F1-score, or any other metric relevant to the use case.

By comparing this metadata, the team can quickly understand that v2 is better because it was trained on more data and used a new informative feature.

1.3.4. Checksums and data integrity

A checksum is a fixed-length alphanumeric value that acts as a unique digital fingerprint for a file. It is calculated using a cryptographic algorithm. If even a single bit changes in the original file, the checksum will change drastically.

Practical example:

1. We versioned the clients.csv file. The CVD tool calculates its hash: f1d2d2f924e986ac86fdf7b36c94bcdf.
2. This hash is stored in the metadata file.
3. When another team member downloads this version, the tool recalculates the hash of the downloaded file.
4. If the hash matches f1d2d2f924e986ac86fdf7b36c94bcdf, there is mathematical certainty that the file is identical to the original, with no corruption during transfer or storage. If it does not match, the tool will alert you to an error, protecting against the use of corrupted data.

1.3.5. Data Provenance vs. Data Lineage

Although often used interchangeably, these terms have nuances.

- **Provenance:** It focuses on the origin and history of a piece of data. It answers the question: "Where did this data come from, and who has accessed it?" It's a concept more closely related to auditing and trust. For example, the provenance of a medical image could include the scanner that captured it, the technician who operated it, and the date.
- **Lineage:** It focuses on flow and transformation. It answers the question: "What technical steps led to the creation of this data?" It's a concept closer to reproducibility and pipeline debugging.

In summary: provenance is the "who" and "where" of the data history, while lineage is the technical "how" and "why." A robust CVD system must capture both.

1.4. Critical application cases

The need for data version control is not abstract; it manifests itself in critical applications where reliability and traceability are non-negotiable.

1.4.1. Machine Learning models in production

A production ML model is a living system. It's not enough to simply deploy it and forget about it. It requires constant monitoring and management, where CVD is essential.

Detailed explanation:

A model is the result of three components: code (the algorithm), data (used for training), and parameters (the training configuration). To reproduce a model, all three are needed. Git manages the code. The CVD manages the data. Configuration files (also versioned in Git) manage the parameters.

- **Model drift management:** Models degrade over time because the real world changes (concept drift). When the performance of a production model declines, the first step is to compare it to previous versions. Is the current model worse than the one from three months ago with the same data? Or is it the new data that's different? CVD allows you to perform these experiments, isolating the cause of the problem.
- **Rollback insurance:** If a newly deployed model turns out to be faulty, it's crucial to be able to quickly and safely revert to the previous version. A rollback involves not only changing the model file but also ensuring that all the infrastructure that consumes it (e.g., inference pipelines) is compatible. CVD, by versioning the model as an artifact along with its lineage, facilitates this process.
- **Comparison of experiments:** A data scientist can train dozens of models by varying the input data (e.g., with or without an external data source). CVD allows each of these experiments to be recorded, associating each resulting model with the exact data that created it and its performance metrics, facilitating rigorous comparison and the selection of the best candidate for production.

1.4.2. Regulatory audits

In sectors such as finance, pharmaceuticals, and any industry subject to regulations such as GDPR, the ability to demonstrate where data comes from and how it has been used is not optional; it is a legal requirement.

Specific examples:

- **GDPR (General Data Protection Regulation):** If a European citizen exercises their "right to be forgotten," the company must be able to demonstrate that it has removed their data from all systems. A clear data lineage is essential to trace where that customer's data has flowed and ensure its complete deletion.
- **Financial sector:** A bank must be able to explain to regulators how the value at risk (VaR) of its portfolio was calculated on a specific date. This requires the model code, the parameters, and, crucially, the exact snapshot of market data from that day.
- **Clinical research:** When submitting a new drug for approval, pharmaceutical companies must

provide a comprehensive record of clinical trial data. Every analysis and conclusion must be traceable back to the original raw data, ensuring that there has been no manipulation.

1.4.3. Reproducible scientific research

The reproducibility crisis is a serious problem in many scientific fields. CVD is one of the most powerful tools to combat it. It allows you to package the code, data, and instructions for running an analysis into a single project, creating an "executable research object." This means that peer reviewers can go beyond simply reading the article; with just a few commands, they can regenerate the results, validating the robustness of the work.

1.4.4. Collaboration in distributed teams

In today's global work environment, it's rare for a data team to be located in the same office. Members may be in different cities or even continents. The CVD provides the necessary infrastructure for efficient, asynchronous collaboration. It acts as a centralized "single source of truth." An analyst in Madrid can process the data and create a new version. A machine learning engineer in Bangalore can take that version, train a model, and record it. The entire change history is transparent, preventing conflicts and ensuring everyone is working with the correct information.

2. VERSIONING STRATEGIES AND METHODOLOGIES

Once the fundamentals of Data Version Control (DVC) have been established and its critical importance understood, the next step is to explore practical strategies and methodologies for its implementation. This section delves into the "how":

- How to name versions consistently.
- How to efficiently store changes.
- How to manage the contextual information that gives meaning to each version.
- How to adapt collaborative software workflows, such as branches and merges, to the world of data.

Adopting a structured approach not only brings technical clarity but also establishes a common language for the team, enabling more robust data governance and smoother collaboration. Decisions made at this stage will have a direct impact on storage costs, team agility, and the organization's ability to audit and reproduce its data assets.

2.1. Semantic versioning adapted to data

Semantic versioning (SemVer) is a widely adopted standard in software development for communicating the nature of changes between versions. Its MAJOR.MINOR.PATCH format provides a clear signal as to whether a new version introduces changes that break compatibility. Adapting this logic to datasets provides a predictable and powerful framework for managing their evolution.

2.1.1. Detailed definition of MAJOR.MINOR.PATCH for datasets

In the context of data, "compatibility" refers to whether a pipeline or script that consumed the previous version can run without errors with the new version.

- **MAJOR (Major change - breaks compatibility):** This number increases when changes are made to the dataset schema that are incompatible with the previous version. Code or analysis that worked with version 1.xx will likely fail or produce incorrect results if run on version 2.0.0 without modifications. These are deep structural changes.
- **MINOR (Minor Change - adds functionality without breaking):** It increments when new data or features (columns) are added in a way that doesn't break backward compatibility. A script that consumed version 1.1.x should be able to process version 1.2.0 without problems, although it might ignore the newly added columns or rows. The changes are additive.
- **PATCH (Patch - fixes bugs without breaking):** This is increased to make corrections to existing data that do not alter the schema or add new features. These are data quality improvements. A script that worked with version 1.1.1 should produce more accurate results with version 1.1.2.

The following are specific examples to illustrate each type of change.

2.1.2. Specific examples of changes

MAJOR changes (vX.yz -> v(X+1).0.0):

1. **Change of data type:** Modify the `id_usuario` column from numeric (INTEGER) to text string (STRING) to accommodate new non-numeric identifiers.
2. **Deleting a column:** Remove the `last_login_date` column because it is considered obsolete or for privacy reasons.
3. **Renaming a column:** Change the name of the revenue column to `gross_revenue_eur` for clarity.
4. **Change file format:** Migrate the CSV dataset to Parquet, which requires a different code reader.
5. **Modifying units without changing the name:** Changing the values in the distance column from kilometers to miles would break all dependent calculations.

Minor changes (v1.Xy -> v1.(X+1).0):

1. **Adding new rows:** incorporate the most recent month's sales data into the historical dataset.
2. **Adding a new (nullable) column:** Add a `discount_code` column that can be empty for past sales.
3. **Data enrichment:** Add demographic columns (e.g., `estimated_age`, `region`) from an external data source.
4. **Inclusion of a new optional field:** In a product JSON, add a new field `technical_specifications` that did not exist before.
5. **Increased granularity:** to move from having daily aggregated data to having hourly data, while maintaining the column scheme.

PATCH changes (v1.1.X -> v1.1.(X+1)):

1. **Correction of calculation errors:** Recalculate the values in the `applied_tax` column due to an error

in the original formula.

2. **Duplicate removal:** Identify and remove 1,000 rows that were exact duplicates.
3. **Correction of typographical errors:** Standardize "Madrid", "madrid" and "Madridd" to a single value in the city column.
4. **Imputation of zero values:** Fill in the NULL values in the height column using the average of your category.
5. **Format standardization:** Unify all date formats in the birth_date column to the YYYY-MM-DD standard.

2.1.3. Borderline cases and how to categorize them

Often, a change may appear to belong to more than one category. The general rule is to choose the category with the greatest impact.

- **Scenery:** Errors in existing data are corrected (PATCH) and, in the same batch, new data from last week is added (MINOR).
Decision: This change should be versioned as a MINOR. Although it includes patches, the addition of new data is a more functionally significant change. The version would change from v1.2.5 to v1.3.0. The commit message or release documentation should detail both actions.
- **Scenery:** A column (MAJOR) is removed, but the column was empty (NULL) in all records.
Decision: Although no information is lost in practice, the schema has changed. This is a MAJOR change. Any script attempting to select that column will fail. The version will change from v1.3.0 to v2.0.0.

2.1.4. Advanced naming conventions

For more complex workflows, SemVer can be extended with additional tags:

- **Pre-release versions:** For experimental or validation data, hyphens and an identifier are added. Examples: v2.1.0-alpha, v2.1.0-beta.2. This indicates that the version is not stable and should not be used in production.
- **Build metadata:** For additional information that does not affect the version, a plus sign (+) is added. Example: v3.0.0+timestamp.20250911. Useful for linking a data version to the specific pipeline run that generated it.

2.1.5. Integration with versioning tools

Semantic versioning is effectively implemented using Git tags. Since tools like DVC link data to code, a Git tag captures the state of the code and the pointers to the data simultaneously. An integrated workflow might look like this:

1. **Data change:** An analyst runs a script that cleans dataset.csv.
2. **Versioned with DVC:** The analyst runs `dvc add dataset.csv`. DVC updates the dataset.csv.dvc file with the new dataset hash.
3. **Commit in Git:** The analyst adds the changes to the Git staging: `git add dataset.csv.dvc` and commits them with a descriptive message: `git commit -m "Fix: Corrects typographical errors in the city column"`.

4. **SemVer's decision:**The change is a data correction. It's a patch. If the previous version was v1.1.1, the new one will be v1.1.2.
5. **Tagged in Git:**An annotated Git tag is created: `git tag -a "v1.1.2" -m "Standardized and corrected city data."`
6. **Synchronization:**The changes and the tag are uploaded to the remote repositories: `dvc push` and `git push --tags`.

This workflow creates an immutable and clearly labeled reference point that locks a semantic version with the exact state of the code and data.

2.2. Versioning strategies

Versioning data involves storing multiple snapshots, which can lead to prohibitive storage costs if the right strategy isn't chosen. The choice depends on the size of the data, the frequency of changes, and access requirements.

2.2.1. Full versioning (Full Snapshotting)

This is the simplest strategy: each time a new version is created, a complete and independent copy of the dataset is saved.

When to use it?:

- With relatively small datasets (up to a few GB).
- When changes are infrequent but significant (e.g., a complete quarterly update).
- In systems where read speed is the highest priority and storage cost is not a limiting factor.

2.2.2. Incremental Versioning (Delta Versioning)

This strategy focuses on storage efficiency. Only the first complete version is saved; subsequent versions are stored as "deltas" or "differentials," which are files that only contain the changes (additions, modifications, deletions) compared to the previous version.

Techniques and reconstruction:

1. **Version v1.0:**The entire base.csv file is stored.
2. **Version v1.1:**The difference between version 1.0 and the new version is calculated. A file named `delta_v1.1.diff` is generated. Only this small delta file is stored.
3. **Reconstruction:**To obtain v1.1, a program must read base.csv and apply the transformations described in `delta_v1.1.diff`.

To speed up access to the latest version, the full last version and backward deltas can be stored. To rebuild an older version, the "reverse" deltas are applied.

When to use it?:

- With massive datasets (terabytes or more) where the cost of storage is the main problem.
- In systems with frequent and small changes (e.g., a database table that is updated every few minutes).
- Data formats such as Delta Lake or Apache Iceberg implement this concept natively.

2.2.3. Hybrid techniques

They combine the best of both worlds to balance cost and performance.

- Implementation:** A full snapshot is stored periodically (e.g., on every major change or once a week) and deltas are used for intermediate versions.


```
s3://mi-bucket/datos/v1.0.0/ (COMPLETE)
s3://my-bucket/data/deltas/v1.0.1.diff
s3://my-bucket/data/deltas/v1.1.0.diff
...
s3://mi-bucket/datos/v2.0.0/ (COMPLETE) <- New starting point
```
- Applications:** This is often the most practical strategy. It limits the length of delta chains, reducing the risk of corruption and shortening rebuild times, without completely sacrificing storage efficiency.

2.2.4. Data deduplication and space optimization

Modern tools like DVC implement an intelligent versioning approach that resembles a hybrid strategy with deduplication. DVC uses content-addressable storage.

- Hashing:** When a file is added (`dvc add my_file`), DVC calculates its checksum (hash).
- Central storage:** The file is moved to a central storage (local or in the cloud), using its hash as the filename.
- Pointer:** In the Git repository, only a small text file (`.dvc`) containing the hash of the original file is saved.

If a file remains unchanged in a new version, its hash will be the same, and DVC will simply reuse the existing file in central storage. It will not be re-uploaded. If the dataset consists of many files, only the new or modified files will be uploaded. This provides automatic file-level deduplication.

2.2.5. Comparison of storage costs

Let's assume a 100 GB dataset that receives 5 GB changes every day for a month (30 days).

Strategy	Storage Day 1	Storage Day 30	Monthly cost (S3 Standard \$0.023/GB))
Complete	100 GB	$100 + (29 * 105) = 3145$ GB (approx)	~ \$72.34
Incremental	100 GB	$100 + (29 * 5) = 245$ GB	~ \$5.64
DVC (Deduplication)	100 GB	$100 + (29 * 5) = 245$ GB	~ \$5.64
Hybrid (Full weekly)	100 GB	$(4 * 100) + (26 * 5) = 530$ GB	~ \$12.19

Note: pReference prices based on AWS S3 Standard rates in effect in 2025. Prices may vary depending on the region and the contracted volume. DVC and incremental versioning have a similar cost in this single-file scenario. The advantage of DVC is its ease of use and file-level management without the need for diff tools.

2.3. Metadata management and traceability

A data version without context is just a collection of bits. Metadata provides that context, explaining what the data is, where it came from, how it was created, and its quality. Traceability, or lineage, connects the versions together, creating a navigable map of the data's evolution.

2.3.1. What metadata should be captured?

Capturing the correct metadata is crucial for governance, reproducibility, and automation. A comprehensive list is provided below for reference:

- **Provenance metadata:**
 - `original_source`: URL, name of the source database or system.
 - `data_owner`: responsible team or person.
 - `version_creation_date`: exact timestamp of when the version was generated.
 - `user_creator`: the ID of the user or service that executed the process.
 - `commit_message`: human-readable description of the change made.
- **Structural metadata (schema):**
 - `column_list`: an array with the names of all the columns.
 - `data_types`: a map of column name to data type (string, int64, float64, datetime).
 - `number_of_columns`: the total number of columns.
 - `restrictions`: information about primary keys, unique values, or non-null columns.
- **Descriptive metadata (statistics):**
 - `row_number`: the total record count.
 - `file_size_bytes`: the size of the file on disk.
 - `statistics_by_column`: for each numeric column: mean, median, standard deviation, minimum, maximum.
 - `frequency_categories`: for each categorical column: count of unique values and frequency of the most common ones.
 - `percentage_nulls`: the percentage of missing values per column.
- **Lineage and reproducibility metadata:**
 - `version_datos_entrada`: the hash or semantic version of the datasets used as input.
 - `commit_git_script`: the Git commit hash of the transformation script that generated this version.
 - `script_name`: the name of the script or notebook (e.g., `cleaning.py`).
 - `execution_parameters`: the arguments with which the script was executed (e.g. `{ "cut-off_date": "2025-09-01" }`).
- **Quality and business metadata:**
 - `result_of_quality_tests`: a summary of the data validations (e.g. `{ "passed": 10, "failed": 1 }`).

- Tags: keywords for search and discovery (e.g., ["sales", "europe", "PII"]).
- description: a natural language explanation of the purpose of the dataset.

2.3.2. Metadata formats

The choice of format depends on human readability and ease of processing by machines.

- **JSON:**

- **Advantages:** universally supported, easy to parse in any language, good for complex nested structures.
- **Example:**

```
{ "version": "v2.1.3", "origin": { "creator_user": "ana.gomez",
"git_script_commit": "a1b2c3d4" }, "statistics": { "number_of_rows": 150234,
"percentage_of_nulls": { "client_id": 0.0, "phone": 12.5 } }}
```

- **YAML:**

- **Advantages:** More human-readable than JSON, allows comments, ideal for configuration files.
- **Example:**

```
version: v2.1.3 origin: user_creator: ana.gomez commit_git_script: a1b2c3d4 #
Commit of the cleanup script
Statistics: Number of rows: 150234 Percentage of nulls: Client ID: 0.0 Telephone:
12.5
```

2.3.3. Automation of metadata capture

Manual metadata capture is prone to errors and does not scale. It should be an automated step within the data pipeline.

Pipelinedata with metadata capture

1. **Activator:** A git push to the main branch activates a Continuous Integration (CI) system such as Jenkins or GitHub Actions.
2. **Pipeline execution (using DVC):** The CI executes the dvc repro command, which in turn executes the transformation script (e.g., process.py).
3. **Metadata generation:** The process.py script, in addition to generating the new dataset, uses a library (e.g., Pandas Profiling) to generate a metadata.json file with all the descriptive and schema statistics.
4. **Lineage capture:** The script also captures CI environment variables (such as the Git commit hash) and adds them to the metadata.json.
5. **Metadata versioning:** The metadata.json file is versioned along with the data using dvc add metadata.json.
6. **Commit and Push:** The CI pipeline commits the new .dvc files and metadata.json.dvc, and uploads them to the repository.

2.3.4. Lineage Tracking Automatic vs. manual

- **Manual:** It involves documenting dependencies in a README.md file or a wiki. It's simple but

fragile, as it easily becomes outdated.

- **Automatic:** Tools like DVC formalize the lineage in a pipeline definition file, typically `dvc.yaml`:

```
stages: process_data: cmd: python scripts/process.py data/raw.csv data/processed.csv
deps: - data/raw.csv - scripts/process.py outs: - data/processed.csv
```

This file explicitly declares that `processed.csv` (an output, out) depends on `raw.csv` and `process.py` (dependencies, deps). This creates a Directed Acyclic Graph (DAG) that DVC can understand. If `raw.csv` or `process.py` changes, DVC knows that `processed.csv` is outdated and needs to be regenerated.

2.4. Branching and merging for datasets

Branching is one of Git's most powerful features, as it allows for parallel work and isolated experimentation. Applying this concept to data is possible, but merging is conceptually very different and more complex than with code.

2.4.1. When should you create branches in data?

1. **Safe experimentation:** A data scientist wants to test a new null imputation technique. They create a branch called `feature/null-imputation`, make the changes, and generate a new version of the data only in that branch. The production dataset in the main branch remains untouched.
2. **Development of new features:** A team is integrating a new data source. They create a `develop` branch where they work on merging and cleaning the new dataset. Only when the data is validated and stable is it merged into `main`.
3. **Bug Fixes (Hotfix):** A serious quality issue was discovered in the production data version `v3.1.4`. A `hotfix/price-correction` branch was created from the `v3.1.4` tag, the problem was fixed, `v3.1.5` was generated, and it was merged back into `main`.

2.4.2. Merge strategies for datasets

Directly merging large binary or CSV data files (as with `git merge`) is not feasible. Instead, higher-level strategies are used.

Strategy 1: Code merging and regeneration (most common and safe)

This is the preferred approach. The conflict is not resolved in the data itself, but in the code that generates it. The flow would be as follows:

1. **Divergence:** Ana creates the branch `feature/eliminar-nulos` and modifies `clean.py`. Bruno creates `feature/imputar-nulos` and also modifies `clean.py`.
2. **Code merging:** Ana's branch is merged into `main` first. Bruno now has to merge the changes from `main` into his branch. Git will detect a conflict in the `clean.py` file.
3. **Code resolution:** Bruno resolves the conflict in the Python script, deciding which cleanup logic to use (or a combination).
4. **Regeneration:** Once the `clean.py` code is merged and there are no conflicts in `main`, the entire pipeline (`dvc repro`) is executed. This script regenerates the final dataset from the original

sources, creating a new version of the data that reflects the merged code logic.

Strategy 2: Merging at the record level (For specific cases)

When two branches modify different and non-overlapping subsets of data.

- **Use case:**The European team works on the feature/data-europe branch and adds clients from Spain. The LATAM team works on feature/data-latam and adds clients from Mexico.
- **Resolution:**The "merge" can be a script that performs a UNION or concatenation operation on the two resulting datasets to create the final version.

2.4.3. Data conflict resolution

A real data conflict occurs when two branches modify the same record with different values.

- **Scenery:**In branch A, the price of product_id=123 is corrected to 50.0. In branch B, it is corrected to 51.5.
- **Resolution:**This cannot be resolved automatically. It requires manual intervention or a predefined policy.
 - **Manual intervention:**A data steward or data owner must be notified to decide what the correct value is.
 - **Resolution policy:**Business rules can be established, such as "always prioritize the value of the pricing system over manual adjustment" or "the most recent change takes priority".

2.5. Retention and archiving policies

It is neither practical nor cost-effective to keep every version of every dataset permanently accessible in high-performance storage. Clear policies are needed to decide what to store, for how long, and where.

2.5.1. Criteria for deciding which versions to keep

- **Production versions:**Any version of data used to train a production model or to generate a regulatory report must be retained indefinitely or as required by law. These versions must be clearly labeled (v1.2.0, report_Q3_2024).
- **Milestone versions:**preserve versions that represent important milestones of the project, such as the end of a research phase or data used for a scientific publication.
- **Recent versions:**Keep all versions from the last N months (e.g., 6 months) in "hot" storage to facilitate rollback and quick comparison.
- **Intermediate versions:**Versions generated in branches of experimental features that were not merged may be deleted after a grace period (e.g., 30 days).

Regulatory issues would need to be taken into account in these types of decisions, such as:

- **GDPR:**It requires that personal data not be kept longer than necessary. Retention policies must reflect this, ensuring the deletion of old data. Data lineage helps demonstrate that a user's data has been removed from all derived versions.
- **SOX (Finance), HIPAA (Health):**They require strict traceability and the ability to audit data over long periods (often 7 years or more).

2.5.2. Long-term archiving strategies

Archiving involves moving old and infrequently accessed data to low-cost, high-latency storage. One possible archiving policy could be:

1. **Monthly evaluation:** An automated script runs on the first day of each month.
2. **Candidate identification:** The script lists all data versions that are older than six months and do not have a "milestone" or "production" label.
3. **Move to cold storage:** Using the cloud provider's API, the script changes the storage class of the data files from S3 Standard to S3 Glacier Deep Archive (AWS example).
4. **Metadata update:** A metadata record is updated to indicate that version v0.5.3 is now archived and its retrieval could take several hours.
5. **Elimination (optional):** For very old versions that exceed the legal retention period, the script can permanently delete them.

2.5.3. Costs vs. benefits of archiving

Factor	Benefit	Cost (or risk)
Monetary cost	Drastic reduction of cloud storage bill (up to 95% savings).	Cost of transition operations and possible cost of early recovery if the data is needed.
Performance	Free up space and improve listing performance on hot storage.	Retrieving archived data is very slow (from minutes to hours), which delays unplanned analysis.
Governance	It promotes active "data hygiene," forcing teams to decide what data is valuable.	Risk of archiving or deleting data that later proves to be valuable. Requires proper cataloging.
Compliance	It helps comply with data retention regulations by explicitly defining lifecycles.	If the policy is incorrect, it can lead to non-compliance by deleting data prematurely.

Implementing these strategies and methodologies transforms data version control from a reactive task into a proactive pillar of an organization's data governance and strategy.

3. VERSIONING TOOLS AND TECHNIQUES IN PRACTICE

After establishing the conceptual and strategic foundations of Data Version Control (DVC), this section moves from theory to practice, exploring the most influential tools and applied techniques that enable data teams to implement robust, reproducible, and collaborative workflows. The goal is to provide a detailed and functional technical guide that serves as a field manual for any professional facing the challenge of managing the data lifecycle.

We'll take an in-depth look at DVC (Data Version Control), the open-source tool that has become an industry standard, detailing its installation, configuration, and daily operation on Windows. We'll also explore alternatives like Git LFS, understand the role of cloud-native solutions, and finally, address when and how a well-structured manual approach can suffice. Each section is designed to be practical, with precise commands, code examples, and solutions to the most common problems you'll encounter. A case study will be presented to illustrate these and other details of versioning and control in practice.

3.1. DVC (Data Version Control)

DVC is an open-source tool that brings Git principles to the world of data and machine learning models. It's not a replacement for Git; it's a layer that works seamlessly with it. While Git manages the code and metadata (small text files), DVC handles the larger files (data, models, etc.), maintaining the connection between them. This synergy allows for the creation of fully reproducible data science projects.

3.1.1. Installation in Windows

Installing DVC on Windows is a simple process. The most common and recommended method is through pip, the Python package manager. It is essential to have Python 3.8 or higher and Git for Windows installed.

Prior Verification:

Before installing, open a Windows terminal (PowerShell or Command Prompt) and check your Python and pip version.:

```
python --version
pip --version
```

To get DVC with support for all types of remote storage (AWS S3, Google Cloud Storage, Azure Blob Storage, SSH, etc.), the best option is to install it with the [all] add-on. This ensures you won't have dependency issues if you later decide to switch cloud providers.

1. Open PowerShell or Command Prompt.
2. Run the general installation command:

```
pip install "dvc[all]"
```

Post-Installation Verification:

Once the installation is complete, verify that DVC is correctly installed and accessible from your PATH by running:

```
dvc --version
```

You should see the version of DVC you just installed, along with information about the supporting libraries and the Windows platform.

```
PS C:\Users\adrian.rico> dvc --version
3.63.0
```

Illustration1DVC installation check

3.1.2. Initial setup

DVC operates within a Git repository. The initial configuration establishes this fundamental relationship in your Windows environment.

Step 1: Create and initialize a Git repository

If you're starting a project from scratch, create a folder and turn it into a Git repository using PowerShell or Command Prompt.

```
# Create a new folder for the project and enter it
mkdir project-dvc-guia

# Initialize the Git repository
git init
```

Step 2: Initialize DVC

Now, within the Git repository, initialize DVC.

```
dvc init
```

This command is the heart of the configuration. It performs several key actions:

- **Create the internal structure of DVC:**It generates a `.dvc/` directory where DVC will store its configuration, data cache, and other internal information.
- **Create configuration files:**like `.dvc/config`.
- **Prepare the integration with Git:**It creates a `.dvcignore` file (similar to `.gitignore`) to make DVC ignore certain files, and most importantly, it prepares Git to not ignore DVC files.
- **Confirmation message:**It will suggest that you add the new files to Git.

Step 3: Make the first commit of the configuration

Follow DVC's recommendation and save the initial state of the project in Git.

```
git add .git commit -m "Initialize DVC"
```

At this point, your project is set up. Git is ready to version the code and metadata, and DVC is ready to version the data. We haven't added any data yet, but the structure is already functional.

3.1.3. All basic commands

Here are examples of the fundamental DVC workflow in daily use, executed from a Windows terminal.

Step 1. dvc add: start versioning a data file

This command tells DVC to start scanning a file or directory.

- **Example:**Let's assume we have a data file `data\clientes.csv`.

```
# Create a folder for the data and a sample file
mkdir dataecho "id,nombre,ciudad" > data\clientes.csvecho "1,Ana,Madrid" >>
data\clientes.csvecho "2,Bruno,Barcelona" >> data\clientes.csv

# Add the file to DVC version control
dvc add data\clientes.csv
```

What has dvc add done?:

1. It has moved the contents of `data\clientes.csv` to the internal DVC cache (`.dvc\cache`).
2. It has created a small metadata file called `data\clientes.csv.dvc`. This file is a plain text pointer that contains the MD5 hash of the original file's contents.
3. He modified the `.gitignore` file to ensure that Git ignores `data\clientes.csv` and does not upload it to the remote code repository.

Step 2. git commit: save the data state

The next step is to save the pointer (file).dvc) in Git. This is how a version of the code is associated with a version of the data.

```
# Check the status of the Git repository
git status

# Add the .dvc file and the modified .gitignore
git adddata\clientes.csv.dvc.gitignore

# Confirm the changes
git commit -m "Add raw client dataset"
```

Now, anyone who clones the repository and sees this commit will know which version of the data was being used.

Step 3. dvc push: Upload the data to remote storage

Until now, the data is only in your local cache. `dvc push` uploads it to the configured remote storage.

```
# (Assuming we have already configured a remote with dvc remote add)
dvc push
```

This command reads the `.dvc` files from the current commit, finds the corresponding data in the local cache, and uploads them to remote storage.

Step 4. `dvc pull`: Download the data from remote storage

This is the command that a collaborator (or you yourself on another machine) would use to obtain the data.

- **Contributor workflow:**
 1. Clone the Git repository: `git clone <repo_url>`
 2. Enter the directory: `cd project-dvc-guide`
 3. Execute `dvc pull`

`dvc pull` reads the `.dvc` files from the current commit, checks if the data is in the local cache and, if not, downloads it from remote storage, placing it in the correct path (e.g., `data\clientes.csv`).

3.1.4. Configuration of different types of remote storage

DVC needs a place to store large files. This storage is separate from where you store your code (GitHub, GitLab, etc.).

```
dvc remote add
```

The ``dvc remote add`` command specifies the location where the data will be stored. This configuration is necessary to proceed with the previously described workflow.

The general syntax is `dvc remote add <remote_name> <remote_url>` and the remote space can be:

1. Local Storage (for testing or small equipment)

You can use a folder on your own file system or on a network drive.

```
# Use a Windows path, outside of your current project
dvc remote add -d my_local_remote D:\dvc-storage
```

2. Amazon S3

You need to have an S3 bucket and AWS credentials configured (usually via AWS CLI with `aws configure`).

```
dvc remote add -d my_s3_remote s3://my-bucket-name/path/to/the/bucket
```

3. Google Cloud Storage (GCS)

You need a bucket in GCS and have authenticated `gcloud` CLI (`gcloud auth application-default login`).

```
dvc remote add -d my_remote_gcs gs://my-bucket-name/path
```

4. Azure Blob Storage

You need a storage account and a container.

```
dvc remote add -d my_azure_remote azure://container-name/path
```

5. SSH

You can use any server you have access to via SSH.

```
dvc remote add -d mi_ssh_remoto ssh:// user@servidor.com :/home/user/dvc-storage
```

Once a remote is configured, DVC modifies the `.dvc/config` file. You should add this change to Git:

```
git add .dvc/configgit commit -m "Configure remote storage"
```

3.1.5. Pipelinesdata with DVC

This is where DVC shines. It allows you to define, execute, and version the steps in a data pipeline, creating an explicit lineage. Pipelines are defined in the `dvc.yaml` file.

Key concept: A pipeline is composed of stages. Each stage has:

- `cmd`: a command to execute.
- `deps`: dependencies (files or directories that are inputs).
- `outs`: outputs (files or directories that are outputs).

If a dependency changes, DVC knows that the stage is "obsolete" and needs to be re-executed.

Example of a simple pipeline: preprocessing

Suppose we have a Python script `src\preprocess.py` that takes `data\clientes.csv` and generates `data\clientes_procesado.parquet`.

```
import pandas as pd
import sys

# Read the command line arguments
input_path = input path
output_path = output path

# Simple preprocessing logic
df = pd.read_csv(input_path)
df['city'] = df['city'].str.upper() # Convert cities to uppercase
df.to_parquet(output_path)
```

The stage is created with `dvc stage add`. This command creates (or updates) the `dvc.yaml` file.

```
dvc stage add -n preprocess ^ -p params.yaml:threshold ^ -d src\preprocess.py -d
data\customers.csv ^ -o data\customers_processed.parquet ^ python src\preprocess.py
data\customers.csv data\customers_processed.parquet
```

The generated `dvc.yaml` file will look like this:

```
stages: preprocess: cmd: python src\preprocess.py data\clientes.csv
data\clientes_procesado.parquet deps: - data\clientes.csv - src\preprocess.py outs: -
data\clientes_procesado.parquet
```

To run the pipeline, simply execute the command ``dvc repro``. This command examines the ``dvc.yaml`` file, checks if any dependencies have changed since the last execution, and if so, re-executes the outdated stages. If nothing has changed, it will do nothing.

3.1.6. Integration with Git workflows

Integration with Git is key to DVC's power. The individual workflow looks like this:

1. **Create a new branch:** A data scientist wants to experiment with a new preprocessing algorithm.
`git checkout -b feature/new-preprocessing`
2. **Modify the code:** Edit `src\preprocess.py` to add new logic.
3. **Play the pipeline:** it is executed `dvc repro`. DVC will detect the change in `src\preprocess.py` and re-

execute the stage, generating a new `data\clientes_procesado.parquet`. `dvc repro` also creates a `dvc.lock` file that fixes the exact versions (hashes) of dependencies and outputs. This file is crucial for reproducibility.

4. **Commit the results:**

```
git add dvc.yaml dvc.lock src\preprocess.py
git commit -m "Feat: Implement new preprocessing logic"
```

5. **Upload the new data:**

```
dvc push
```

6. **Upload the code:**

```
git push origin feature/new-preprocessing
```

The experiment is now isolated on its own branch. Another team member can review it simply by running ``git checkout feature/new-preprocessing`` and then ``dvc pull``.

3.1.7. Troubleshooting of common problems

Some common problems and their possible solutions are:

1. **Problem:**ERROR: failed to push data to the cloud - The specified bucket does not exist
 - **Symptom:**dvc push fails with an authentication or resource not found error.
 - **Probable cause:**The cloud credentials are not configured in the environment, or the bucket/container name in `.dvc/config` is incorrect.
 - **Solution:**Verify that your CLI (AWS, gcloud, az) is correctly configured and authenticated. Check the `.dvc/config` file to ensure the remote URL is correct.
2. **Problem:**dvc repro does nothing, even though I've changed the data.
 - **Symptom:**You run `dvc repro` but DVC says that everything is up to date.
 - **Probable cause:**You haven't used `dvc add` to register the change in the data. DVC only detects changes in the files it crawls.
 - **Solution:**run `dvc add <modified_file>` before `dvc repro`.
3. **Problem:**I accidentally added a large file to Git.
 - **Symptom:**git push takes forever or is rejected by the server due to file size.
 - **Probable cause:**You forgot to add the file to `.gitignore` before doing `git add ..`
 - **Solution:**Use tools like `git-filter-repo` or `BFG Repo-Cleaner` to remove the file from the Git history. Then, add it back with ``dvc add`` and commit the `.dvc` file.
4. **Problem:**ERROR: file has no checksum
 - **Symptom:**dvc pull fails because it cannot find a hash in the cache or on the remote.
 - **Probable cause:**The `.dvc` file you are trying to download was created incorrectly, or someone did `dvc push` without doing `dvc add` first.
 - **Solution:**Go back to the original commit where the data was added and make sure the `dvc add -> git commit -> dvc push` flow was executed correctly. You may need to regenerate the output and force a new push.
5. **Problem:**DVC performance is slow with thousands of small files.
 - **Symptom:**Commands like `dvc status` or `dvc push` take a long time.

- **Probable cause:**DVC calculates a hash for each file. With many small files, the overhead is considerable.
 - **Solution:**instead of `dvc add data\images\`(which tracks each image individually), groups the files into a single .zip or .tar file and versions that single file: `dvc add data\images.zip`.
6. **Problem:**Cache directory is growing too large
- **Symptom:**the directory `.dvc\cache` takes up a lot of disk space.
 - **Probable cause:**The cache contains data versions of old branches or discarded experiments.
 - **Solution:**Use ``dvc gc`` to clear the cache. Make sure you have a backup if you want to keep data from branches that aren't in your local repository.

3.2. Git LFS - An alternative for large files

Git LFS (Large File Storage) is a Git extension developed by GitHub for handling large files. To use it on Windows, you must first install Git for Windows, and then run `git lfs install` only once.

Like DVC, it doesn't store large files directly in the Git repository. Instead, it saves a small text file with a pointer, and the large file is uploaded to a separate storage area.

The main difference with DVC is philosophical and functional:

- **Git LFS** It is a Git extension, designed to make versioning large files almost transparent.
- **DVC** It is a data science tool, designed to understand and manage pipelines and experiments, that uses Git as part of its workflow.

3.2.1. When to use Git LFS vs DVC?

The choice between DVC and Git LFS depends entirely on the use case. They are not direct competitors in all scenarios.

Feature	Git LFS	DVC	Conclusion
Main purpose	Version large files transparently within Git.	Manage the complete lifecycle of data and ML projects (data, code, pipelines, metrics).	Use Git LFS If you only need to version a few large assets (e.g., videos, PSDs) and don't have a complex pipeline, use DVC for any serious data science project.

Pipeline awareness	Null. Doesn't know how a file was generated. Only knows that it's a large file.	Fundamental. DVC builds a dependency graph (DAG) and knows which code generates which data.	DVC is vastly superior for pipeline reproducibility. Git LFS offers no help in this regard.
Storage	Tightly coupled with the Git provider (GitHub, GitLab, etc.). Uses its LFS storage.	Completely agnostic. You can use S3, GCS, Azure, SSH, or your own server. You have full control over where your data resides.	DVC offers much more flexibility and control over the costs and location of data storage.
Granularity	Version individual files.	You can version files, directories, and pipeline stage outputs.	DVC is more granular and better suited to data project structures.
Ease of use	Very easy for Git users. Just type <code>`git lfs track "*.psd"`</code> and then use Git normally.	Slightly steeper learning curve, as it introduces new concepts such as pipelines and remotes.	Git LFS is simpler for simple tasks. DVC is more powerful but requires learning its specific workflow.
Ideal use case	A graphic designer creating versions of Photoshop files. A video game developer creating versions of 3D textures.	A data scientist training a churn model. A BI team processing ETL reports. A researcher who needs their analysis to be reproducible.	The tools are designed for different audiences and problems.

3.3. Native cloud solutions

Major cloud providers offer native versioning capabilities in their object storage services. These powerful tools can be a viable alternative, especially if you're already deeply embedded in a particular cloud ecosystem. Unlike DVC or Git LFS, they don't integrate directly with Git; instead, they operate at the bucket or container level.

3.3.1. AWS S3 with native versioning

- **Description:**By enabling versioning on an S3 bucket, whenever an object is overwritten or deleted, S3 doesn't delete it, but instead creates a new version and preserves the old one. Each version has a unique ID. Any previous version can be listed, restored, or permanently deleted.
- **Key functionality:**Object Lock and retention for compliance.

3.3.2. Google Cloud Storage (GCS) versioning

- **Description:**Similar to S3, GCS object versioning, when enabled on a bucket, saves an "outdated" version whenever an object is replaced or deleted. These older versions can be listed and managed.
- **Key functionality:**lifecycle policies to automatically remove or change the storage class of older versions (e.g., move versions older than 90 days to Coldline Storage).

3.3.3. Azure Blob Storage options

- **Description:** Azure offers two main mechanisms:
 1. **Versioning:** It automatically creates new versions of a blob every time it is modified. It is the option most similar to S3 and GCS.
 2. **Snapshots:** These are read-only snapshots of a blob at a given moment, which must be created manually. They are useful for point backups.
- **Key functionality:** immutability at the account or container level for compliance requirements.

3.3.4. Comparison of costs and features

Feature	AWS S3	Google Cloud Storage	Azure Blob Storage
Storage cost	You are charged for each version stored. Costs can add up quickly if not managed.	Similar to S3, each version takes up space and is billed.	Each version or snapshot is billed as a normal blob (although snapshots can be more efficient if there are only deltas).
Cost of operations	Charges apply to version requests (GET, PUT, LIST). Restoring typically incurs no additional cost beyond the request fee.	Operating charges apply. Lifecycle policies may have associated operating costs.	Version and snapshot operations have costs.
Automatic management	Robust lifecycle policies to expire or transition older versions to cheaper storage (Glacier).	Highly flexible and easy-to-configure lifecycle policies.	Lifecycle management policies for moving or deleting older versions.
Ease of use	Relatively simple to enable, but managing version IDs can be complex without tools.	Easy to enable and manage via the console or gsutil.	Versioning is straightforward; snapshots require more active management.

3.3.5. Integration with Machine Learning tools

These native versioning solutions integrate seamlessly with the same vendor's ML platforms. For example:

- **AWS SageMaker** It can directly read specific versions of S3 objects, allowing you to associate a training run with a specific dataset version ID.
- **Google Vertex AI** It integrates natively with GCS, allowing Vertex pipelines to use versioned datasets.
- **Azure Machine Learning** It uses Azure Blob Storage datastores, and you can target specific

snapshots or versions to ensure reproducibility of training jobs.

The main disadvantage is that lineage (the connection between code and data version) must be managed manually or with the help of ML platform tools, which is often less explicit and portable than the solution offered by DVC.

3.4. Manual versioning and conventions

Not all projects need the power (and overhead) of a tool like DVC. For small, single-researcher projects, or in very restrictive environments, a disciplined manual versioning system may suffice. The key is consistency.

3.4.1. When is manual versioning appropriate?

- **Solo projects:** when there is no need to collaborate or resolve conflicts.
- **Dataset small and stable:** if the data changes infrequently and its size is manageable.
- **Initial exploration phases:** during rapid prototyping, where the formality of a CVD system can slow down the process.
- **Restrictive corporate environments:** where it is not possible to install open source tools like DVC.

3.4.2. Robust naming conventions

A good naming convention is the cornerstone of manual version control. It should be informative and easy to organize. A logical structure might be:

[DATASET_NAME]_[ISO_DATE]_[SEMVER_VERSION]_[STATE].ext

- **[DATASET_NAME]:** descriptive base name (e.g., sales_customers).
- **[ISO_DATE]** YYYYMMDD or YYYY-MM-DD. Allows you to sort files chronologically.
- **[VERSION_SEMVER]:** vMAJOR.MINOR.PATCH (e.g., v1.2.0). Follows the logic of semantic versioning.
- **[STATE]:** optional, but useful (e.g. raw, cleaned, final, dev).
- **.ext:** file extension (e.g., .csv, .parquet).

Examples:

- sales_historical_20250915_v1.0.0_raw.csv (Initial raw data)
- sales_historical_20250916_v1.1.0_cleaned.parquet (Cleaned data with new columns)
- clients_segmentation_20250917_v2.0.0_final.csv (Final version with changes to the schema)

3.4.3. Templates for manual documentation

Along with the versioned data, there should be a text file (ideally Markdown) that documents the changes.

Example of CHANGELOG.md:

```
# Dataset Changelog: Historical Sales

## v1.1.0 - 2025-09-16

### Added- New enrichment columns: `geographic_region`, `acquisition_channel`. - Sales data for the first half of September 2025.

### Changes - Migrated the format from CSV to Parquet to improve performance.

## v1.0.1 - 2025-09-15

### Fixed - 5,200 duplicate transactions have been removed. - Typographical errors in the `product_name` column have been corrected.

## v1.0.0 - 2025-09-15

- Initial version of the dataset.
- Contains sales data from 2020-01-01 to 2025-08-31.
```

This approach, although manual, provides a basic level of traceability and governance that is infinitely better than having no system at all.

4. PRACTICAL CASE STUDY

This section is a practical "minimum effort, maximum result" guide that allows you to understand 80% of the DVC workflow used on a daily basis.

The philosophy is simple: start with a local setup, without cloud dependencies, and using generated data. Once the user masters this workflow, scaling to more complex environments becomes much more intuitive.

The work cycle described in this section is as follows:

1. INITIAL CYCLE

- Prepare environment (Git, DVC, Python).
- Generate data (Python script) - Generates sales_store.csv.
- Version data (dvc add) - Creates .dvc/cache and ventas_tienda.csv.dvc.
- Versioning code and metadata (git commit) - Saves to Git history.
- Synchronize data (dvc push) - Uploads data to the local remote storage.

2. UPDATE CYCLE (Repetitive)

- Update data (Python script) - Adds new rows to the CSV.
- Go back to step 3 and repeat: dvc add → git commit → dvc push.

4.1. Initial setup

First, the working environment is prepared. Minimal tools and a clean directory structure are required. All commands are designed for a PowerShell terminal on Windows.

4.1.1. Minimum prerequisites

The following must be installed beforehand:

1. **Git**: the version control system for DVC code and metadata. [Download Git](#).
2. **DVC**: the tool for versioning data. It is installed with pip.
3. **Python** To run the scripts for generating and modifying data, the pandas and numpy libraries are required.

Open a terminal and run:

```
pip install dvc pandas numpy
```

We will also create the logical folder structure that allows us to keep the project organized.

```
# Create the project's main folder and navigate to it
mkdir sales-project; cd sales-project

# Create subfolders for data and scripts
mkdir data
mkdir scripts
```

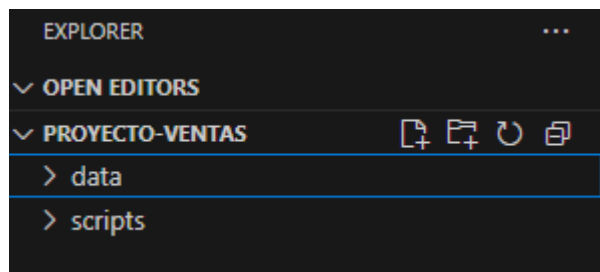


Illustration2: visualization in VS Code of the created folders

4.1.2. Initializing repositories

Next, Git and DVC are initialized in the project root.

```
#Git initialization
git init

#DVC Initialization
dvc init
```

```

> ▾ TERMINAL
A ● PS C:\Users\adrian.rico\OneDrive - NEORIS\Desktop\proyecto-ventas> git init
  Initialized empty Git repository in C:/Users/adrian.rico/OneDrive - NEORIS/Desktop/proyecto-ventas/.git/
● PS C:\Users\adrian.rico\OneDrive - NEORIS\Desktop\proyecto-ventas> dvc init
  Initialized DVC repository.

  You can now commit the changes to git.

  +-----+
  | DVC has enabled anonymous aggregate usage analytics. |
  | Read the analytics documentation (and how to opt-out) here: |
  | <https://dvc.org/doc/user-guide/analytics> |
  +-----+

  What's next?
  -----
  - Check out the documentation: <https://dvc.org/doc>
  - Get help and share ideas: <https://dvc.org/chat>
  - Star us on GitHub: <https://github.com/iterative/dvc>
● PS C:\Users\adrian.rico\OneDrive - NEORIS\Desktop\proyecto-ventas>

```

Illustration3: git and dvc initialization results in the project

4.1.3. Local remote storage configuration

For this example, "remote storage" will be another folder on the same computer.

```

# Create a folder for DVC storage outside the project
mkdir ../dvc-storage-simple

# Configure DVC to use that folder as the default remote folder
dvc remote add -d local_storage ../dvc-storage-simple

```

```

● PS C:\Users\adrian.rico\OneDrive - NEORIS\Desktop\proyecto-ventas> mkdir ../dvc-storage-simple

  Directory: C:\Users\adrian.rico\OneDrive - NEORIS\Desktop

  Mode                LastWriteTime         Length Name
  ----                -
  d-----           29/09/2025    13:50         dvc-storage-simple

● PS C:\Users\adrian.rico\OneDrive - NEORIS\Desktop\proyecto-ventas> dvc remote add -d local_storage ../dvc-storage-simple
  Setting 'local_storage' as a default remote.

```

Illustration4: creation of external storage and assignment to the project

With these steps, the environment is ready to begin versioning data.

4.2. First versioning of data

In this step, the first set of data is generated and versioned.

4.2.1. Creation and execution of the data generator script

A very simple Python script is created. This script will generate a CSV file with test data that will be used in the case. This script must be created in the project's scripts folder.

```
# scripts/generate_data.py
import pandas as pd
import os

# Data file path DATA_PATH = "data/data.csv"

# Initial data content
initial_data = { 'id': [1, 2, 3], 'value': [100, 150, 200]}

# Create the DataFrame and save it as CSV
df = pd.DataFrame(initial_data)

# Ensure that the 'data' directory exists
os.makedirs(os.path.dirname(DATA_PATH), exist_ok=True)

df.to_csv(DATA_PATH, index=False)

print(f"File '{DATA_PATH}' generated with 3 records.")
```

Next, the script is run to create the data.csv file.

```
python scripts/generate_data.py
```

● Archivo 'data/datos.csv' generado con 3 registros.

Illustration5: result of the execution of the data generation script

4.2.2. Version control using DVC and GIT

First, DVC is instructed to crawl the data file.

```
dvc add data/datos.csv

# Add all new and modified files
git add .

# Create the commit
git commit -m "feat: Initial data version (v1.0)"
```

DVC creates the file data/datos.csv.dvc (a pointer to the data) and adds data/datos.csv to the .gitignore file. The changes are then saved to Git.

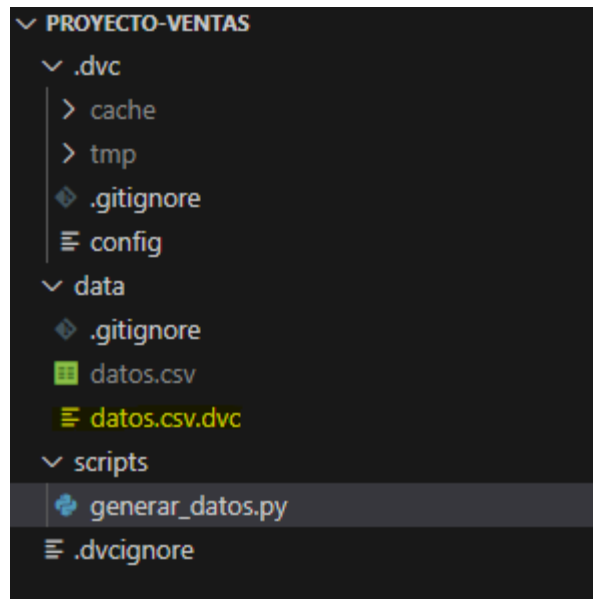


Illustration6: Example of the creation of the .dvc file for data tracking

4.2.3. Documentation and synchronization with the remote

Following the best practices discussed in section 3, we create a markdown file that collects the changes, adds it to git, and synchronizes the data from storage to the local remote that was configured in point 4.1.

```
# CHANGELOG.md## [1.0] - 2025-09-29

### Added- Initial version of the dataset `datos.csv`.- Contains 3 records with columns
`id` and `valor`.

#Add changelog.md to git

git add CHANGELOG.mdgit commit -m "docs: Add initial CHANGELOG"

#New changes are synchronized with the remote

dvc push
```

```

PS C:\Users\adrian.rico\OneDrive - NEORIS\Desktop\proyecto-ventas> git add CHANGELOG.md
PS C:\Users\adrian.rico\OneDrive - NEORIS\Desktop\proyecto-ventas> git commit -m "docs: Anadir CHANGELOG inicial"
[master 243b248] docs: Anadir CHANGELOG inicial
1 file changed, 5 insertions(+)
create mode 100644 CHANGELOG.md
PS C:\Users\adrian.rico\OneDrive - NEORIS\Desktop\proyecto-ventas> dvc push
Collecting [1.00 [00:00, 250entry/s]
Pushing
1 file pushed
PS C:\Users\adrian.rico\OneDrive - NEORIS\Desktop\proyecto-ventas>

```

Illustration7: results of synchronization with remote storage

4.3. Update simulation

Next, an update of the dataset is simulated, adding new records.

4.3.1. Creation and execution of the update script

A second script is created that adds data to the generated file.

```

# scripts/update_data.py
import pandas as pd
import os

DATA_PATH = "data/datos.csv"

# Data to add
new_data = { 'id': [4, 5], 'value': [250, 300]}

df_nuevo = pd.DataFrame(nuevos_datos)

# Add to existing file without writing header
df_new.to_csv(DATA_PATH, mode='a', header=False, index=False)

print(f"2 records have been added to '{DATA_PATH}'")

```

Next, the script is executed and it is verified that the file now has a length of 6 (1 header plus 5 rows of data)

```

python scripts/actualizar_datos.py

# The result should be 6 (1 header + 5 rows of data)(Get-Content data/datos.csv).Length

```

```

Se han añadido 2 registros a 'data/datos.csv'.
PS C:\Users\adrian.rico\OneDrive - NEORIS\Desktop\proyecto-ventas> (Get-Content data/datos.csv).Length
6

```

Illustration8: result of the execution of the data update script

4.3.2. Versioning of data updates and control updates

The process for versioning changes is identical to the previous one.

1. **Add to DVC.**
2. **Update CHANGELOG.md.**
3. **Confirm in Git.**
4. **Sync with the Remote.**

```
#The new version of the data is added to the tracking
dvc add data/datos.csv

#Markdown is updated
## [1.1] - 2025-09-29

### Added- 2 new records to the dataset.

#The changes are committed to git and the data is synchronized on the remote
git add data/datos.csv.dvc CHANGELOG.mdgit commit -m "feat: Add new data (v1.1)"
dvc push
```

```
PS C:\Users\adrian.rico\OneDrive - NEORIS\Desktop\proyecto-ventas> dvc add data/datos.csv
100% Adding...| 1/1 [00:00, 8.58file/s]

To track the changes with git, run:

    git add 'data/datos.csv.dvc'

To enable auto staging, run:

    dvc config core.autostage true
PS C:\Users\adrian.rico\OneDrive - NEORIS\Desktop\proyecto-ventas> git add data/datos.csv.dvc CHANGELOG.md
PS C:\Users\adrian.rico\OneDrive - NEORIS\Desktop\proyecto-ventas> git commit -m "feat: Anadir nuevos datos (v1.1)"
[master 87f08d2] feat: Anadir nuevos datos (v1.1)
2 files changed, 8 insertions(+), 2 deletions(-)
PS C:\Users\adrian.rico\OneDrive - NEORIS\Desktop\proyecto-ventas> dvc push
Collecting
Pushing
1 file pushed
```

Illustration9: results of the data update versioning

4.4. Recovery and rollback

If an update contains errors, DVC allows you to easily revert the data to a previous version.

Scenery The last two records added are incorrect and you need to revert to v1.0.

The workflow is as follows:

1. Identify the previous version: Use git log to view the history. The commit before HEAD is HEAD~1.
2. Perform the double checkout: the rollback process requires two steps:
 - **Git checkout:** the .dvc pointer file from the previous version is recovered.

- **DVC checkout** The data is synchronized to match that pointer.

```
# 1. Retrieve the pointer from the previous commit
git checkout HEAD~1 data/data.csv.dvc

# 2. Synchronize the data file
dvc checkout data/datos.csv

#3. Verification

#It is verified that the data file has returned to its original state.

# The result should be 4, as in the initial version(Get-Content data/datos.csv).Length
```

```
PS C:\Users\adrian.rico\OneDrive - NEORIS\Desktop\proyecto-ventas> git checkout HEAD~1 data/datos.csv.dvc
Updated 1 path from d91503f
PS C:\Users\adrian.rico\OneDrive - NEORIS\Desktop\proyecto-ventas> dvc checkout data/datos.csv
Building workspace index |2.00 [00:00, 125entry/s]
Comparing indexes |3.00 [00:00, 3.00kentry/s]
Applying changes |1.00 [00:00, 63.7file/s]
M data/datos.csv
PS C:\Users\adrian.rico\OneDrive - NEORIS\Desktop\proyecto-ventas> (Get-Content data/datos.csv).Length
4
```

Illustration10: results of reverting the file to its original state

If the rollback is permanent, a new commit should be created in Git to save this state.

```
git add data/datos.csv.dvc
git commit -m "revert: Revert to v1.0 data due to errors"
```

4.5. Conclusion

This case study completes the basic data versioning cycle with DVC in a local, controlled environment. It demonstrates how to set up a project from scratch, systematically version datasets, update data while maintaining a complete history, and revert changes when necessary. This minimalist, yet simple, workflow encapsulates the fundamental principles of data version control developed throughout this guide.

The key to success in implementing a data versioning system lies not only in the choice of tool or technical configuration, but also in the discipline and best practices adopted by the team. As discussed in previous sections, this includes:

- The use of semantic versioning to clearly communicate the nature of the changes.
- Comprehensive documentation for each version in files such as CHANGELOG.md.
- Automatic capture of metadata that provides context and traceability.
- Seamless integration between Git and DVC to keep code and data synchronized.

By consistently following these practices, any organization, regardless of size or complexity, can establish a robust, auditable, and reproducible data management system that will serve as a solid foundation for advanced analytics, machine learning, and scientific research projects.

5. BEST PRACTICES AND IMPLEMENTATION

The successful implementation of a data version control system depends not only on the choice of technical tools but also on the adoption of consistent and sustainable organizational practices. This final section summarizes the essential governance recommendations, phased adoption processes, and fundamental resources that will facilitate the transition from informal systems to professional data lifecycle management. The goal is to provide a practical and scalable framework that teams can implement gradually, starting with basic conventions and evolving toward more sophisticated processes as their specific needs evolve.

5.1. Fundamental conventions and standards

The first step toward a robust versioning system is to establish clear naming and documentation conventions. These conventions should be simple enough to avoid friction in daily work, yet rigorous enough to ensure traceability. For version names, it is recommended to adopt data-adapted semantic versioning (MAJOR.MINOR.PATCH), where each component clearly communicates the nature of the change. File names should follow predictable patterns that include the dataset name, the date in ISO format (YYYY-MM-DD), and the semantic version.

Commit messages associated with data changes should follow a consistent structure that includes: the type of change (add, modify, correct), a concise description of what was modified, and the justification for the change. For example: "PATCH: Corrected null values in temperature column - 15 records with impossible values (<-100°C) were detected and imputed using the 24-hour moving average."

This structure greatly facilitates subsequent auditing and understanding of the change history. Supporting documentation should be maintained in a CHANGELOG.md file that records each released version with its date, changes made, and potential impact on existing analyses.

5.2. Governance roles and processes

Effective governance requires clearly defined responsibilities, even in small teams. It is recommended to establish at least three fundamental roles:

- **Data Owner**, responsible for strategic decisions regarding the dataset and its evolution.
- **Data Steward**, responsible for the operational management of versioning and quality assurance.
- **Data Consumers**, users who consume the published versions for their analysis.

In small teams, one person can assume multiple roles, but the conceptual distinction helps to clarify responsibilities.

Review and approval processes should be adapted to the type of change. Patch changes (minor fixes) may only require a technical quality review, while major changes (those that break compatibility) must undergo a more thorough review process that includes assessing the impact on existing pipelines and models, as well as providing advance notice to all affected customers. It is recommended to establish a simplified emergency procedure for critical situations where serious errors need to be corrected immediately, always followed by a post-mortem review.

5.3. Compliance and retention considerations

Version retention policies must balance regulatory requirements with storage costs. As a general rule, all versions used for production models or regulatory reports should be retained indefinitely or as required by applicable regulations (typically 7–10 years in the financial and healthcare sectors). Development and experimental versions can be archived or deleted after a reasonable grace period (30–90 days). For organizations subject to personal data processing regulations, it is critical to establish procedures that document how personal data is managed through versioning, including the ability to remove information about specific individuals from all versions when the right to be forgotten is exercised.

We can see a summary of the best practices of the framework discussed in the following table:

Area	Recommended practice
Nomenclature	Use semantic versioning (vMAJOR.MINOR.PATCH) + ISO date in filenames
Documentation	Keep CHANGELOG.md updated with each released version.
Commits	Descriptive messages with the following format: TYPE + description + justification
Revision	Major changes require an impact review; patch changes require only a technical review.
Retention	Retain production versions indefinitely; archive development versions after 90 days
Compliance	Document complete lineage for audits; procedure in accordance with current disposal regulations

5.4. Roadmap progressive implementation

Adopting a data version control system should be gradual and pragmatic. A phased approach is recommended to allow for rapid value creation and learning from each stage before scaling.

Phase 1 (Weeks 1-4): Foundation. Select a critical dataset as a pilot, establish basic naming conventions, implement DVC with local or simple cloud storage, and train the team on essential commands (add, commit, push, pull, checkout). The goal is to achieve the first versioned version and for the team to become familiar with the basic workflow.

Phase 2 (Weeks 5-12):Expansion. Extend versioning to the 3-5 most critical datasets, implement basic DVC pipelines to capture data lineage, establish simple review procedures, and create documentation templates. This phase consolidates practices and begins to generate tangible benefits in reproducibility.

Phase 3 (Weeks 13-24):Maturity. Implement branching for experiments, automate metadata capture using scripts, integrate versioning with CI/CD pipelines, and establish formal retention and archiving policies. At this stage, the system is fully operational and the team has mastered advanced workflows.

Phase 4 (Months 6+):Optimization. Evaluate usage and efficiency metrics, optimize storage costs through hybrid strategies, scale the system across the entire organization, and implement automated monitoring and alerts. This phase focuses on continuous improvement and enterprise scalability.

5.5. Tool decision matrix

Choosing the right tool depends on multiple contextual factors:

- For small teams (1-5 people) working with datasets up to 10 GB, a manual versioning system with strict naming conventions may be sufficient.
- For medium-sized teams (5-20 people) with datasets up to 100 GB and active collaboration needs, DVC is the recommended option due to its balance between power and simplicity.
- For large organizations with massive datasets (>100 GB) and enterprise requirements, cloud-native solutions (S3 Versioning, GCS, Azure Blob) integrated with ML platforms may be more appropriate. Git LFS is a viable alternative for small datasets (<5 GB) when the team is already very familiar with Git and prefers a minimal learning curve.

Tool	Equipment size	Data size	Ideal use case
Manual + Nomenclature	1-5 people	<10 GB	Small projects, initial exploration
Git LFS	3-10 people	<5 GB	Teams with Git experience, compact datasets
DVC	5-50 people	10 GB - 10 TB	Data science, ML, active collaboration
Native Cloud (S3/GCS/Azure)	20+ people	>100 GB	Enterprise, integration with ML platforms

6. CONCLUSION

Effective data management has become a strategic imperative for any organization that aspires to extract real value from its information assets. Throughout this guide, we have explored how Data Version Control (DVC) has evolved from a desirable practice to a fundamental requirement for ensuring reproducibility, traceability, and effective collaboration in data and machine learning projects. What began as an adaptation of software development principles has transformed into its own discipline, with specialized tools, mature methodologies, and clearly defined use cases ranging from scientific research to regulatory compliance.

The journey from chaotic manual versioning, characterized by files with names like `final_v2_NOW_YES.csv`, to robust systems based on tools like DVC, represents much more than a mere technical improvement. It represents a paradigm shift in how organizations conceptualize and manage their data. Data is no longer simply static files copied and shared via email; it is dynamic assets with a complete lifecycle, a documented lineage, and a history that must be preserved and accessible. This transformation not only improves operational efficiency but also mitigates critical risks: from the inability to reproduce scientific results to failing regulatory audits that can lead to multimillion-dollar fines and irreparable damage to corporate reputation.

Throughout the previous chapters, we have established that data version control solves fundamental problems that directly impact an organization's ability to innovate and compete. The "ghost model," the "impossible audit," and "collaborative chaos" are not hypothetical scenarios, but everyday realities in data teams lacking structured versioning systems. The costs stemming from these problems—measured in lost time, flawed business decisions, and legal penalties—more than justify the investment in implementing a data version control system. Beyond the cost lies the positive value: the ability to rapidly iterate on machine learning models, confidently revert to previous versions when something goes wrong, collaborate seamlessly across globally distributed teams, and ensure that every analysis, every report, and every model deployed to production can be fully reproduced and audited.

The strategies and methodologies detailed in the second section, from data-adapted semantic versioning to retention and archiving policies, provide a comprehensive and flexible framework that can be adapted to the specific needs of each organization.

The practical implementation, explored in depth through DVC, demonstrates that the barrier to entry for adopting a version control system is surprisingly low. DVC, in particular, stands out for its elegant design, which makes it a natural extension of Git, allowing teams to leverage their existing knowledge of code version control. DVC's ability to define and execute reproducible pipelines, automatically capturing data lineage, elevates version control to a new level: not only is the state of the data versioned, but also the process that led to that state. This feature is invaluable for data science and machine learning, where reproducibility is not just a best practice, but a requirement for scientific validation and reliability in production.

However, as explored in section five, technology is only one part of the equation. The success of implementing such a system depends fundamentally on people, processes, and organizational culture. An organization may have the most advanced technical infrastructure, but if data scientists lack the discipline to make descriptive commits, if data engineers don't document changes in a changelog, or if project leaders don't establish clear governance policies, the system will quickly degrade. Adopting consistent naming conventions, clearly defining roles and responsibilities, and establishing review processes commensurate with the risk of each change are just as critical as the technical configuration of the tools.

Implementing a robust data version control system is a journey, not a destination. It requires commitment, investment, and, above all, a cultural shift that recognizes data for what it truly is: the

most valuable and sensitive asset of the digital age. With the tools, methodologies, best practices, and implementation frameworks outlined in this guide, any organization—from a startup to a multinational, from a research lab to a financial institution—now has the knowledge to embark on this journey with confidence. The path is laid out, the tools are available, the practices are documented, and the benefits are clearly demonstrated. The next step is yours.

DO YOU WANT TO KNOW MORE ABOUT
THE **WHAT DOES THIS INITIATIVE CONTRIBUTE?**

Visit <https://datos.gob.es>

Instagram: [@datosgob](#)

Twitter: [@datosgob](#)

LinkedIn: [datos.gob.es](#) Subscribe to our [bulletin](#)

Write to contacto@datos.gob.es

You can identify open data
spaces thanks to this logo.



**datos
abiertos**